

Pioneer 3-AT em ROS 2 Humble + Gazebo Harmonic

Guia de instalação e execução (passo a passo)

Projeto `ros-pioneer3at` — migração ROS 2

3 de junho de 2026

Resumo

Este documento descreve, passo a passo, como instalar e executar a simulação do robô **Pioneer 3-AT** em **ROS 2 Humble** com **Gazebo Harmonic** (gz-sim 8). Ao final, você terá o robô em um mundo simulado, controlável pelo teclado, com odometria, transformadas (TF) e um sensor LiDAR publicados para o ROS 2 e visualizados no RViz2. Este pacote é uma migração do projeto original em ROS 1 + Gazebo Classic; o código antigo permanece em `legacy_ros1/`.

Sumário

1	Pré-requisitos	1
2	Instalação	1
2.1	Instalar as dependências	1
2.2	Clonar o repositório	2
2.3	Criar o workspace e compilar	2
3	Executando a simulação	2
3.1	Simulação completa (Gazebo + RViz)	2
3.2	Dirigir o robô pelo teclado (teleop)	2
3.3	Teste rápido sem teclado	3
3.4	Visualizar apenas o modelo (sliders das juntas)	3
4	Tópicos principais	3
5	Estrutura do projeto	3
6	Solução de problemas	4
7	Trabalho futuro	4

1 Pré-requisitos

- **Sistema operacional:** Ubuntu 22.04 (ou base equivalente, ex. Zorin OS 17).
- **ROS 2 Humble** (`ros-humble-desktop`).
- **Gazebo Harmonic** (gz-sim 8.x).
- `git` e `colcon` para clonar e compilar.

Confirme as versões antes de começar:

```
source /opt/ros/humble/setup.bash
ros2 --version      # deve indicar Humble
gz sim --version    # deve indicar 8.x (Harmonic)
```

2 Instalação

2.1 Instalar as dependências

Instale os pacotes de integração ROS↔Gazebo e os utilitários usados pelo projeto:

```
sudo apt update
sudo apt install \
  ros-humble-ros-gz-sim \
  ros-humble-ros-gz-bridge \
  ros-humble-xacro \
  ros-humble-robot-state-publisher \
  ros-humble-joint-state-publisher-gui \
  ros-humble-teleop-twist-keyboard \
  ros-humble-rviz2
```

2.2 Clonar o repositório

Clone o projeto e selecione a branch da migração ROS 2. Ajuste a URL para o seu repositório/-fork:

```
# escolha uma pasta de trabalho qualquer
cd ~/Documentos
git clone <URL-DO-REPOSITÓRIO> ros-pioneer3at
cd ros-pioneer3at
git checkout ros2-humble-harmonic-migration
```

2.3 Criar o workspace e compilar

O ROS 2 compila pacotes dentro de um *workspace* colcon (uma pasta com um subdiretório **src/**). Vamos criar um workspace e colocar este pacote dentro dele por meio de um *symlink* (atalho), de forma que as pastas de build fiquem fora do repositório:

```
mkdir -p ~/ros2_ws/src
ln -s ~/Documentos/ros-pioneer3at ~/ros2_ws/src/pioneer3at

cd ~/ros2_ws
colcon build --packages-select pioneer3at
```

Sempre que abrir um terminal novo para usar o projeto, **carregue o ambiente** (ROS 2 + workspace):

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
```

Dica: para não repetir isso toda vez, adicione as duas linhas acima ao final do seu `~/.bashrc`.

3 Executando a simulação

3.1 Simulação completa (Gazebo + RViz)

Sobe, em um único comando, o Gazebo Harmonic com o mundo, o robô, a ponte ROS↔Gazebo e o RViz2:

```
ros2 launch pioneer3at simulation.launch.py rviz:=true
```

Sem o RViz (apenas Gazebo + ponte):

```
ros2 launch pioneer3at simulation.launch.py
```

Argumentos disponíveis:

Argumento	Padrão	Descrição
world	pioneer_world.sdf	arquivo .sdf do mundo
rviz	false	abre o RViz2
use_sim_time	true	usa o relógio da simulação

3.2 Dirigir o robô pelo teclado (teleop)

Com a simulação aberta, abra **outro terminal**, carregue o ambiente (Seção 2.3) e rode:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Mantenha essa janela em foco e use as teclas indicadas na tela. As principais:

Tecla	Ação
i	andar para frente
,	andar para trás
j	girar para a esquerda
l	girar para a direita
k	parar
q / z	aumentar / diminuir a velocidade

Os comandos saem no tópico `/cmd_vel`, são levados ao Gazebo pela ponte e movem o robô.

3.3 Teste rápido sem teclado

Para verificar o movimento sem o teleop, publique uma velocidade diretamente:

```
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist '{linear: {x: 0.3}}'
```

O robô deve andar para frente. Pressione Ctrl+C para parar de publicar.

3.4 Visualizar apenas o modelo (sliders das juntas)

Para inspecionar a montagem do robô *sem* o Gazebo, usando sliders manuais para girar as rodas:

```
ros2 launch pioneer3at display.launch.py
```

Abrem-se o RViz2 (com o modelo) e a janela do `joint_state_publisher_gui` (com 4 sliders, um por roda). Arraste os sliders e confira que cada roda gira em torno do eixo correto e que todos os links estão conectados na árvore de TF.

4 Tópicos principais

Com a simulação rodando, liste os tópicos com `ros2 topic list`. Os principais:

Tópico	Tipo	Sentido
/cmd_vel	geometry_msgs/msg/Twist	ROS → Gazebo
/odom	nav_msgs/msg/Odometry	Gazebo → ROS
/tf	tf2_msgs/msg/TFMessage	Gazebo → ROS
/joint_states	sensor_msgs/msg/JointState	Gazebo → ROS
/scan	sensor_msgs/msg/LaserScan	Gazebo → ROS
/clock	roscpp_msgs/msg/Clock	Gazebo → ROS

Exemplos de inspeção:

```
ros2 topic echo /odom      # odometria
ros2 topic echo /scan      # leitura do LiDAR
ros2 topic echo /clock     # taxa do relógio de simulacao
```

5 Estrutura do projeto

```
urdf/pioneer3at.urdf.xacro # descricao do robo (visual + colisao + inercia +
  plugins)
worlds/pioneer_world.sdf   # mundo do Gazebo Harmonic (chao, luz, obstaculo)
config/ros_gz_bridge.yaml  # mapeamento de topicos ROS <-> Gazebo
config/pioneer3at.rviz     # config do RViz para a simulacao (frame odom)
config/pioneer3at_display.rviz # config do RViz para o display (frame base_link)
launch/
  simulation.launch.py      # bringup completo (gazebo + spawn + bridge + rviz)
  gazebo.launch.py         # so o Gazebo + mundo
  spawn_robot.launch.py    # robot_state_publisher + spawn no gz
  bridge.launch.py         # so a ponte ros_gz_bridge
  display.launch.py        # so RViz com o modelo + sliders (sem Gazebo)
meshes/                    # malhas STL (reaproveitadas do projeto original)
legacy_ros1/               # codigo ROS 1 / Gazebo Classic original (referencia)
```

6 Solução de problemas

O robô não aparece no Gazebo / malhas não carregam. Verifique se o ambiente está carregado (Seção 2.3) e recompile. O launch define automaticamente `GZ_SIM_RESOURCE_PATH` para que o Gazebo resolva os caminhos `package://pioneer3at/...`

Avisos “Detected jump back in time” e o robô pisca no RViz. Ocorre quando o RViz inicia antes do relógio de simulação (/clock) estar disponível. O `simulation.launch.py` já mitiga isso iniciando o RViz alguns segundos depois do Gazebo. Confirme também que o RViz usa tempo de simulação: `ros2 param get /rviz2 use_sim_time` deve retornar `True`.

`display.launch.py` abre, mas o robô não aparece. Esse modo usa o frame fixo `base_link` (não há odom sem o Gazebo). Garanta que está usando a config `pioneer3at_display.rviz` (já é o padrão do launch).

Altere arquivos do pacote e nada mudou. Recompile e recarregue o ambiente:

```
cd ~/ros2_ws && colcon build --packages-select pioneer3at
source install/setup.bash
```

7 Trabalho futuro

A base de simulação já publica `/odom`, `/tf` e `/scan`, que são os pré-requisitos para:

- **Navegação autônoma** com o **Nav2**.
- **Mapeamento (SLAM)** com o **slam_toolbox**.

Esses recursos substituem o antigo `move_base` / `gmapping` / `amcl` do projeto em ROS 1.